Paper Organization. The rest of this paper is organized as follows. Section 2 describes our data curation, code generation, and feature extraction. Section 3 and Section 4 present our analysis of code differences and detection performance. Section 5 examines feature importance and generalization patterns. Section 6 discusses implications, while Section 7, Section 8, and Section 9 address limitations, validity threats, and related work. Section 10 concludes the paper.

2 DATASET

To compare human-written code with code generated by LLMs, we need a data source containing code already authored by human programmers so that we can generate corresponding code using different LLMs for the same tasks. In this section, we explain how we choose our data source, generate corresponding code from four contemporary LLMs, and ensure fair cross-model comparison through careful data curation.

2.1 Data Source

As mentioned before, we break our analyses into two levels: function-level and class-level. In the following, we explain how we prepare the dataset for these two levels of code.

2.1.1 Function-level: In this work, we choose CodeSearchNet [55] as our data source for function-level code. Widely adopted by prior research [14, 45, 47, 78, 84, 91, 107, 119], this corpus enables us to study real-life development through over two million functions mined from open-source GitHub repositories across six programming languages. We specifically utilize the dataset's *(comment, code)* pairs, where *code* denotes the human-written function body and *comment* denotes the corresponding top-level docstring, as illustrated in Listing 2.

Listing 1. Example of a standalone function from pysubs2 [1] with its docstring.

```
def timestamp_to_ms(groups):
    """
    Convert groups from :data:`pysubs2.time.TIMESTAMP` match to milliseconds.
    Example:
    >>> timestamp_to_ms(TIMESTAMP.match("0:00:00.42").groups())
    420
    """
    h, m, s, frac = map(int, groups)
    ms = frac * 10**(3 - len(groups[-1]))
    ms += s * 1000
    ms += m * 60000
    ms += h * 3600000
    return ms
```

CodeSearchNet has over 500, 000 Python functions with docstrings. For our study, we focus exclusively on standalone functions. These functions have no dependencies on other functions or classes within the same module. We adopt this constraint because, through preliminary experimentation, we observed that when functions have contextual dependencies (e.g., calling other module-level functions or referencing class attributes), older LLMs (such as GPT-3.5 and Claude 3 Haiku) often fail to generate complete implementations and instead return placeholder code or incomplete snippets. This issue is not unique to our study; Xu *et al.* [111] reported similar challenges when generating code with contextual dependencies. From the available standalone functions, we randomly sample 20, 000 functions to limit processing time and cost associated with code generation across multiple LLMs.

2.1.2 Class-level: Functions and classes represent fundamentally different software artifacts with distinct structural characteristics. For example, functions encapsulate algorithmic logic while classes define data structures and behavioural interfaces, which may lead to different generation patterns. Examining both granularities allows us to determine whether detection strategies must be tailored